

Requisitos de Segurança RA3 - EducaFlix

Documento de requisitos de segurança atualizados do software final

ID	Categoria	Origem	Requisito	Justificativa	Status	Local no código	Como demonstrar na defesa	Teste relacionado
RS-01	DFD/STRIDE - Spoofing	Análise de autenticação	O sistema deve autenticar usuários antes de permitir acesso a funcionalidades protegidas.	Reduz risco de uso indevido de contas e funções privadas.	Implementado	backend/src/middlewares/authMiddleware.js backend/src/routes/index.js	Acessar GET /api/me sem cookie e mostrar HTTP 401 depois fazer login e acessar com cookie.	backend/tests/security.test.js
RS-02	DFD/STRIDE - Tampering	Análise de manipulação de dados	O sistema deve validar dados recebidos antes de criar ou alterar vídeos e avaliações.	Impede persistência de dados incompletos, fora de faixa ou com formato inválido.	Implementado	backend/src/validators/schemas.js backend/src/middlewares/validateRequests.js backend/src/routes/index.js	Tentar criar vídeo com URL inválida ou duração zero e mostrar HTTP 400.	backend/tests/security.test.js
RS-03	DFD/STRIDE - Information Disclosure	Análise de exposição de dados	O sistema não deve retornar senha, hash ou salt nas respostas da API.	Evita vazamento de credenciais mesmo quando endpoints de cadastro, login ou admin retornam usuários.	Implementado	backend/src/repositories/userRepository.js backend/src/controllers/authController.js backend/src/controllers/adminController.js	Fazer cadastro/login e mostrar que a resposta possui usuário sem senhaHash e sem senhaSalt.	backend/tests/security.test.js
RS-04	Codificação segura	OWASP Password Storage Cheat Sheet	O sistema deve armazenar senhas com hash criptográfico e salt, nunca em texto puro.	Protege senhas em caso de acesso indevido ao arquivo de persistência.	Implementado	backend/src/services/passwordService.js backend/src/controllers/authController.js backend/src/database/sqliteDatabase.js	Abrir o banco backend/data/database.sqlite após cadastro e comparar que senhaHash difere da senha original pela camada de leitura da aplicação.	backend/tests/security.test.js
RS-05	Codificação segura	OWASP ASVS	O sistema deve proteger rotas privadas com autenticação, autorização e CSRF em operações mutáveis.	Reduz acesso indevido e requisições forjadas em rotas com alteração de estado.	Implementado	backend/src/middlewares/authMiddleware.js backend/src/middlewares/adminMiddleware.js backend/src/routes/index.js backend/src/services/sessionService.js	Mostrar que usuário comum recebe HTTP 403 ao tentar criar vídeo e que POST/PUT/DELETE exigem X-CSRF-Token.	backend/tests/security.test.js
RS-06	Codificação segura	CWE-89 / OWASP	O sistema deve evitar injeção em consultas usando consultas parametrizadas na camada de repositório.	Reduz risco de injeção e mantém o acesso ao SQLite centralizado no backend.	Implementado	backend/src/repositories/userRepository.js backend/src/repositories/videoRepository.js backend/src/repositories/reviewRepository.js backend/src/database/sqliteDatabase.js	Apontar os prepare(...).run/get/all com parâmetros e mostrar que as entradas continuam validadas antes do acesso ao banco.	backend/tests/security.test.js
RS-07	Livre	Boas práticas de configuração	O sistema deve carregar configurações sensíveis e de ambiente por variáveis, com exemplo versionado.	Evita segredos reais no repositório e facilita execução local/CI.	Implementado	.env.example backend/.env.example backend/src/middlewares/securityHeaders.js backend/src/database/sqliteDatabase.js .gitignore	Mostrar .env.example e .gitignore ignorando .env e o arquivo SQLite local.	Não aplicável
RS-08	Livre	Validação de entrada	O sistema deve validar campos obrigatórios no cadastro, login, vídeos, avaliações e status de usuário.	Reduz erros de negócio e entradas inconsistentes antes da camada de persistência.	Implementado	backend/src/validators/schemas.js backend/src/middlewares/validateRequests.js	Enviar payload sem campos obrigatórios ou com senha fraca e mostrar HTTP 400.	backend/tests/security.test.js
RS-09	Livre	Controle de acesso por propriedade	O sistema deve impedir que um usuário altere ou remova avaliação de outro usuário.	Preserva integridade e privacidade de dados vinculados a conta.	Implementado	backend/src/controllers/reviewController.js backend/src/repositories/reviewRepository.js	Criar avaliação com um usuário e tentar alterar com outro, esperando HTTP 403.	backend/tests/security.test.js
RS-10	Livre	Qualidade e automação	O sistema deve possuir testes automatizados para autenticação, validação, autorização e entidade principal.	Garante regressão mínima para os requisitos de segurança exigidos na RA3.	Implementado	backend/tests/security.test.js .github/workflows/tests.yml package.json backend/package.json	Rodar npm test localmente e mostrar o workflow Tests no GitHub Actions.	backend/tests/security.test.js